Relatório de Teste de Mutação

Projeto operacoes-mutante | StrykerJS + Jest

Campo	Informação
Disciplina	Testes de Software
Trabalho	Análise e melhoria de suíte por teste de mutação
Aluno(a)	Amanda Bueno Campos Peixoto
Data	10/05/2026
Repositório GitHub	https://github.com/abc-peixoto/operacoes-mutante

1. Análise inicial

A suíte inicial possuía 50 testes, um para cada função exportada. A cobertura de código parecia alta: 85,41% de statements, 58,82% de branches, 100% de funções e 98,64% de linhas. Apesar disso, a primeira execução do Stryker mostrou uma pontuação de mutação bem menor: 73,71% no score total e 78,11% considerando apenas código coberto.

Métrica	Resultado inicial
Cobertura de linhas	98,64%
Cobertura de funções	100,00%
Cobertura de branches	58,82%
Mutation score	73,71%
Mutantes mortos / timeout	154 mortos + 3 timeout
Mutantes sobreviventes	44
Mutantes sem cobertura	12

A discrepância ocorreu porque cobertura mede se uma linha foi executada, mas não mede se a asserção é forte o bastante para detectar uma mudança de comportamento. Muitos testes verificavam apenas o caminho feliz, como números positivos, casos verdadeiros e arrays ímpares. Assim, mutações em limites, ramos falsos e mensagens de erro continuavam produzindo resultados aceitos pela suíte original.

2. Análise de mutantes críticos

Foram selecionados três mutantes da primeira execução por representarem falhas típicas da suíte: ausência de casos negativos, ausência de testes de fronteira e ausência de cobertura em ramos específicos.

StrykerJS Mutation Test Report Survived

Mutante crítico 1: isPrimo

src/operacoes.js:74 | Arithmetic/Conditional mutation | status inicial: Survived

Original

```
for (let i = 2; i < n; i++) { if (n % i === 0) return false; }
```

Mutado pelo Stryker

```
for (let i = 2; i >= n; i++) { if (n % i === 0) return false; }
```

Por que sobreviveu?

A suíte original verificava apenas isPrimo(7) como verdadeiro. Sem um número composto, a alteração que impedia o laço de executar não era percebida.

Figura 1. Captura/reprodução do mutante sobrevivente na primeira execução.

StrykerJS Mutation Test Report Survived

Mutante crítico 2: isMaiorQue

src/operacoes.js:104 | EqualityOperator | status inicial: Survived

Original

```
function isMaiorQue(a, b) { return a > b; }
```

Mutado pelo Stryker

```
function isMaiorQue(a, b) { return a >= b; }
```

Por que sobreviveu?

O teste original usava apenas 10 > 5. Para esse caso, as duas versões retornam true. Faltava o limite a === b.

Figura 2. Captura/reprodução do mutante sobrevivente na primeira execução.

StrykerJS Mutation Test Report Survived

Mutante crítico 3: medianaArray

src/operacoes.js:111-112 | Conditional/Arithmetic mutation | status inicial: Survived/NoCoverage

Original

```
if (sorted.length % 2 === 0) return (sorted[mid - 1] + sorted[mid]) / 2;
```

Mutado pelo Stryker

```
if (false) return sorted[mid];
```

Por que sobreviveu?

A suíte inicial só usava array ímpar e ordenado. O ramo de tamanho par e o comportamento de ordenação não eram validados.

Figura 3. Captura/reprodução do mutante sobrevivente na primeira execução.

No caso de isPrimo, a mutação no laço sobreviveu porque a suíte só testava um número primo. Sem isPrimo(4), o teste não demonstrava que números compostos deveriam retornar false. Em isMaiorQue, trocar > por >= não altera o resultado quando a entrada é 10 e 5; o caso que revela o erro é a igualdade. Em medianaArray, os testes originais não passavam por arrays pares nem verificavam ordenação, deixando mutantes nesse ramo sobreviverem ou ficarem sem cobertura.

3. Solução implementada

A melhoria foi feita adicionando testes pequenos e direcionados, cada um desenhado para falhar caso o código mutado fosse executado. Os novos casos não substituem os testes originais; eles complementam a suíte com fronteiras, ramos falsos e caminhos de erro.

- Mensagens e caminhos de erro: validação de exceções em divisão por zero, raiz negativa, fatorial negativo, arrays vazios e inverso de zero.
- Fronteiras de funções: fatorial(0), fatorial(1), raizQuadrada(0) e mediaArray([]).
- Booleanos em ambos os sentidos: isPar(101), isImpar(8), isDivisivel(10, 3), comparações falsas e igualdade em isMaiorQue/isMenorQue.
- Teoria dos números e limites: isPrimo(1), isPrimo(4), produtoArray([]), clamp abaixo, acima e exatamente nos limites.
- Operadores aritméticos mascarados por zero: conversões com 100 C e 212 F.
- Mediana: arrays desordenados e arrays pares, como [7, 1, 3, 5], cujo resultado esperado é 4.

Alguns mutantes restantes eram equivalentes: por exemplo, remover o atalho de fatorial(0) e fatorial(1) ainda leva o laço a retornar 1; produtoArray([]) também retorna 1 pelo reduce com valor inicial; e clamp(valor === min/max) retorna o mesmo número pelo ramo mutado ou pelo retorno final. Esses casos foram marcados com comentários específicos do Stryker, indicando o motivo técnico para ignorá-los.

4. Resultados finais

Métrica	Resultado final
Testes Jest	56 testes passando
Cobertura	100% statements, branches, funções e linhas
Mutation score	100,00%
Mutantes mortos / timeout	199 mortos + 3 timeout
Mutantes sobreviventes	0
Mutantes sem cobertura	0

A meta de pontuação de mutação superior a 98% foi atingida. O relatório final do Stryker foi gerado em reports/mutation/mutation.html e apresenta 100,00% de mutation score.

5. Conclusão

O teste de mutação mostrou que uma suíte pode ter cobertura alta e ainda assim ser fraca. A cobertura inicial indicava que quase todas as linhas eram executadas, mas o Stryker revelou que muitos testes não tinham asserções capazes de distinguir o comportamento correto de pequenas alterações. Ao adicionar casos de fronteira, ramos negativos e entradas mais representativas, a suíte passou a validar melhor a intenção do código. Por isso, teste de mutação é uma ferramenta importante para avaliar a qualidade dos testes, não apenas a quantidade de código percorrido.